Name of Course	Bachelor of Computer Application (BCA)
Semester	V
Name of Subject	Web Development and PHP Programming
Subject Code	BCA-502
Marks	75 Marks
Lectures	50 Lectures

Unit I

1. Introduction to PHP

1.1 Introduction to PHP , History and Features of PHP
1.2 Installation & Configuration of PHP
1.3 Embedding PHP code in Your Web Pages
1.4 Understanding PHP, HTML, and White Space. Writing Comments in PHP
1.5 Sending Data to the Web Browser
1.6 Data types in PHP, Keywords in PHP
1.7 Using Variables, Constants in PHP
1.8 Expressions in PHP
1.9 Operators in PHP

1.1 Introduction to PHP , History and Features of PHP:

Introduction to PHP:

PHP (Hypertext Preprocessor) is a widely-used **open-source server-side scripting language** especially suited for web development. It is embedded within HTML and is used to create dynamic web pages and applications.

History of PHP:

1994: Created by **Rasmus Lerdorf** to track visits to his online resume.
1995: Released as **Personal Home Page Tools (PHP Tools)**.
1997: PHP 3 was released, officially named **PHP: Hypertext Preprocessor**.
2000 onward: Major versions (PHP 4, 5, 7, and 8) were introduced with improved features and performance.

Features of PHP:

1. **Open Source –** Free to use and has a large community.
2. **Cross-Platform –** Runs on various operating systems like Windows, Linux, macOS.
3. **Database Support –** Works well with MySQL, PostgreSQL, Oracle, etc.
4. **Embedded in HTML –** PHP code can be easily inserted within HTML.
5. **Server-side Execution –** Processes data on the server and sends output to the browser.

1.2 Installation & Configuration of PHP:

To run PHP scripts, we need a **web server**, **PHP**, and optionally a **database** like MySQL. These can be installed separately or together using packages like **XAMPP**, **WAMP**, or **MAMP**.

1. Using XAMPP (Windows/Linux/Mac):

Download XAMPP from the official website (apachefriends.org).
Run the installer and follow the setup instructions.
XAMPP installs **Apache (web server)**, **PHP**, and **MySQL** together.
After installation, open the **XAMPP Control Panel** and start **Apache** and **MySQL**.

2. Manual Installation (Windows Example):

Download PHP from the official website (php.net).
Install a **web server** (like Apache or Nginx).
Configure the server to work with PHP by editing config files (e.g., `httpd.conf` for Apache).
Set environment variables (like adding PHP to system PATH).

3. Configuration of PHP:

The main configuration file is **`php.ini`**.
You can modify settings like:

`upload_max_filesize` – for file uploads
`memory_limit` – maximum memory a script can use
`error_reporting` – level of error messages shown

After changes, **restart the server** to apply them.

4. Testing Installation:

Create a file called **`test.php`** with the code:

php

```
<?php phpinfo(); ?>
```

Save it in the web directory (e.g., `htdocs` in XAMPP).
Open a browser and go to `http://localhost/test.php`.

1.3 Embedding PHP code in Your Web Pages:

PHP can be easily embedded within HTML to create **dynamic web pages**. This means PHP code is written inside an HTML file using special tags.

1. PHP Tags:

PHP code is placed between the `<?php` and `?>` tags.

Example:

php

```
<?php
    echo "Hello, World!";
?>
```

2. Basic HTML with Embedded PHP:

html

```
<!DOCTYPE html>
<html>
<head>
```

```
        <title>PHP Example</title>
    </head>
    <body>
        <h1>Welcome!</h1>
        <p>
            <?php
                echo "Today is " . date("l");
            ?>
        </p>
    </body>
    </html>
```

In this example, PHP generates the current day dynamically.

3. File Extension:

Save the file with a `**.php**` extension (e.g., `index.php`).
PHP code will only be executed on a server with PHP installed.

4. Output:

The browser only sees the **HTML output** of the PHP code.
Any logic, calculation, or database interaction is done **server-side**.

5. Importance:

Embedding PHP allows for **dynamic content**, like showing usernames, dates, or data from a database.

1.4 Understanding PHP, HTML, and White Space. Writing Comments in PHP:

1. PHP and HTML Together:

PHP is usually embedded inside HTML to create dynamic web pages.
HTML handles structure and layout, while PHP handles logic and data.

Example:

```
    php
        <html>
        <body>
            <h1>
                <?php echo "Welcome to my website!"; ?>
            </h1>
        </body>
        </html>
```

2. White Space in PHP:

White space (spaces, tabs, newlines) in PHP is generally **ignored** by the parser.
It is used to **improve readability** and format code clearly.

Example:

```
    php
```

```
<?php
    $x = 5;
    $y = 10;
    echo $x + $y;
?>
```

The spaces and newlines do not affect how the code runs.

Writing Comments in PHP:

3 Types of Comments:

1. Single-line comment (using //):

php

```
// This is a single-line comment
```

2. Single-line comment (using #):

php

```
# Another way to write a single-line comment
```

3. Multi-line comment (using /* ... */):

php

```
/*
    This is a multi-line comment
    spanning multiple lines
*/
```

Comments are used to explain code, making it easier to understand and maintain.

1.5 Sending Data to the Web Browser:

In PHP, **sending data to the web browser** means displaying output (like text, HTML, or variables) that the user can see on the webpage.

1. Using `echo` Statement:

`echo` is the most common way to send output.

php

```
<?php
    echo "Hello, World!";
?>
```

This sends **"Hello, World!"** to the browser.

2. Using `print` Statement:

`print` works similarly to `echo`, but returns a value (1).

php

```
<?php
    print "Welcome to PHP!";
?>
```

3. Sending HTML with PHP:

You can send HTML tags using `echo`:

php

```
<?php
    echo "<h1>This is a heading</h1>";
?>
```

This will display a heading in the browser.

4. Sending Variables:

PHP can send variable data to the browser:

php

```
<?php
    $name = "Ali";
    echo "Hello, " . $name;
?>
```

5. Importance:

Sending data is essential for displaying results, messages, or dynamic content like user names, product lists, etc.

1.6 Data types in PHP, Keywords in PHP:

1. Data Types in PHP:

PHP supports different data types to store various kinds of values:

a. Scalar Data Types:

Integer – Whole numbers (e.g., `10`, `-5`)
Float (Double) – Decimal numbers (e.g., `3.14`, `-2.5`)
String – Sequence of characters (e.g., `"Hello"`)
Boolean – True or false (`true`, `false`)

b. Compound Data Types:

Array – Collection of values (e.g., `$arr = [1, 2, 3];`)
Object – Instance of a class

c. Special Data Types:

NULL – Represents no value
Resource – Reference to external resources like database connections

2. Keywords in PHP:

Keywords are **reserved words** that have special meaning in PHP.
They **cannot be used as variable names**, function names, or identifiers.

Examples of PHP Keywords:

`if`, `else`, `while`, `for`, `function`, `return`, `echo`, `class`, `public`, `private`, `var`, `try`, `catch`, `new`, `switch`, `case`, `default`, `break`, `continue`, `true`, `false`, `null`

Example:

php

```
<?php
    $name = "Ali"; // string
    $age = 25;     // integer
    echo "Name: " . $name;
?>
```

1.7 Using Variables, Constants in PHP:

1. Variables in PHP:

Variables are used to **store data** that can change during script execution.
In PHP, variables start with a `**$**` **symbol**.

Rules for variables:

Must start with a letter or underscore (`_`)
Cannot start with a number
Are **case-sensitive**

Example:

php

```
<?php
    $name = "Ali";
    $age = 20;
    echo "Name: " . $name;
?>
```

2. Constants in PHP:

Constants are **fixed values** that **do not change** during script execution.
Use the `**define()**` function to create a constant.
Constants **do not have a `$` symbol**.

Example:

php

```
<?php
    define("SITE_NAME", "MyWebsite");
    echo SITE_NAME;
?>
```

Key Differences:

Feature	Variable	Constant
Symbol	Starts with `$`	No symbol
Changeable	Yes	No
Defined by	Assignment(`=`)	`define()` function

1.8 Expressions in PHP:

An **expression** in PHP is any combination of **variables, values, operators, and functions** that produces a result or value.

1. Types of Expressions:

a. Arithmetic Expressions:

Used to perform mathematical operations.

php

```
<?php
    $x = 10;
    $y = 5;
    $result = $x + $y; // 15
?>
```

b. String Expressions:

Used to combine strings.

php

```
<?php
    $greeting = "Hello " . "World!";
?>
```

c. Logical Expressions:

Used to make decisions (returns true or false).

php

```
<?php
    $x = 10;
    $y = 5;
    $check = ($x > $y) && ($y > 0); // true
?>
```

d. Assignment Expressions:

Assigns values using `=`.

php

```
<?php
    $a = 20;
?>
```

2. Importance of Expressions:

Expressions are the **building blocks** of PHP code.

They are used in **calculations, conditions, loops, and function calls**.

3. Example:

php

```
<?php
    $price = 100;
```

```
        $tax = 0.1;
        $total = $price + ($price * $tax);
        echo "Total price: " . $total;
?>
```

1.9 Operators in PHP:

Operators are symbols used to perform operations on variables and values. PHP supports several types of operators for tasks like arithmetic, comparison, logic, assignment, etc.

1. Types of Operators:

a. Arithmetic Operators:

Used for basic math operations.

Operator	Description	Example
`+`	Addition	`$a + $b`
`-`	Subtraction	`$a - $b `
`*`	Multiplication	`$a * $b `
`/`	Division	`$a / $b `
`%`	Modulus (Remainder)	`$a % $b `

b. Assignment Operators:

Used to assign values to variables.

Operator	Example	Meaning
`=`	`$a = 5`	Assigns 5 to $a
`+=`	`$a += 2`	$a = $a + 2
`-=`	`$a -= 3`	$a = $a - 3

c. Comparison Operators:

Used to compare values (returns true or false).

Operator	Description	Example
`==`	Equal to	`$a == $b`
`!=`	Not equal to	`$a != $b `
`>`	Greater than	`$a > $b `
`<`	Less than	`$a < $b `
`>=`	Greater or equal	`$a >= $b `

d. Logical Operators:

Used in conditional statements.

Operator	Description	Example
`&&`	AND	`($a > 5 && $b < 10)`
`||`	OR	`($a > 5 || $b < 10) `
`!`	NOT	`!($a == $b) `

2. Example:

php

```
<?php
    $x = 10;
    $y = 5;
    if ($x > $y && $x != 0) {
        echo "Valid numbers";
    }
?>
```